

Lab0 Git 实验报告

郭怡琳 25300680194

一、Git 基本用法与思考题

1. 你之前有过多人协同开发的经历吗？如果有，你们是使用什么方式分工协作的？

我在参加AI Hackthon比赛中有过多人协同开发的经历，主要依托魔搭平台进行。协作过程中，每个人会将自己的代码上传到平台并设置为公开，其他成员可以在已有代码的基础上继续修改和完善。这种方式能够实现代码共享，但不同成员实际修改的通常是各自基于原始代码产生的副本，成员之间的修改不会自动合并到同一份代码历史中。

2. Git 为什么要设计“暂存—提交”两个步骤？

Git 将一次修改正式写入版本历史之前，有暂存区这一层。因为一次真实开发过程中，经常同时存在很多不同性质的修改。如果直接`git commit`自动把所有东西保存进去，会得到一个状态混乱的大commit。

而有了`git add`则允许开发者精确选择哪些修改进入下一次提交。当工作区中同时存在多个不同目的的修改时，可以选择其中一部分进入本次提交，使每个 commit 对应一个相对完整、明确的修改任务。这样不仅有助于保持版本历史清晰，也方便后续进行代码审查、问题定位和版本回退。

3. `git branch` 和 `git branch -a` 的区别是什么？

`git branch` 主要用于查看本地分支，当前所在的分支会使用 `*` 标记。例如：

```
* main
  feature
```

其中 `main` 和 `feature` 都属于本地分支。

`git branch -a` 中的 `-a` 表示 `--all`。除了本地分支之外，它还会显示远程跟踪分支，例如：

```
remotes/origin/main
```

其中 `origin` 是远程仓库的默认名称，`origin/main` 表示本地 Git 对远程仓库 `main` 分支状态的记录。

二、模板仓库建立与第一次提交

1. 使用模板仓库建立个人仓库

根据实验要求，我进入课程提供的 `ICS-26Fall-FDU/GitLab` 模板仓库，选择：

```
Use this template
```

→ Create a new repository

在自己的 GitHub 账号下建立个人实验仓库 **lab0-Gitlab**。

使用模板仓库可以直接获得课程预先准备好的文件和目录结构，之后的所有修改和提交均发生在自己的仓库中，不会影响原模板仓库。

2. 克隆仓库到本地

在完成 GitHub SSH 认证配置后，在 WSL 中执行：

```
git clone git@github.com:<username>/lab0-Gitlab.git
cd lab0-Gitlab
git status
git branch
```

此时显示：

```
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

这说明当前位于 **main** 分支，本地仓库与 GitHub 上的远程仓库状态一致，并且工作区中不存在尚未提交的修改。

3. 修改 **main.c**

在 VSCode 中打开本地仓库，根据实验要求完成 **main.c** 文件中的 **TODO** 部分。

修改并保存文件后执行：

```
git status
```

Git 检测到：

```
modified:   main.c
```

此时修改仅存在于工作区，还没有进入暂存区。

4. 暂存、提交并推送

首先执行：

```
git add main.c
```

将 `main.c` 的修改加入暂存区。

随后执行

```
yilin@LAPTOP-ET6G27C4:~/lab0-Gitlab$ git commit -m "Complete"
[main 1818a06] Complete
1 file changed, 1 insertion(+)
```

最后执行

```
yilin@LAPTOP-ET6G27C4:~/lab0-Gitlab$ git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 18 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 300 bytes | 300.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local
objects.
To github.com:linlin-1262/lab0-Gitlab.git
a40c8e9..1818a06 main -> main
```

将本地 `main` 分支的新提交同步到 GitHub 远程仓库。

三、阅读材料与“为什么要学习 Git”

1. 《Commit message 和 Change log 编写指南》

这篇文章主要介绍了规范化 Commit Message 的意义和基本格式。规范的提交信息可以使项目历史更加清晰，方便开发者快速判断每次修改的目的，也有助于按类型筛选提交以及自动生成 Change Log。

一种常见的 Commit Message 格式为：

```
<type>(<scope>): <subject>

<body>

<footer>
```

其中 Header 是最核心的部分，`type` 可以表示本次提交的性质。

本次实验中，我第一次提交时使用的是：Complete 虽然能够正常完成 Git 提交，但仅从提交信息本身很难判断究竟完成了什么。如果按照较规范的方式，可以写为：feat: complete TODO in main.c Commit Message 不只是一次提交的简单备注，它本身也是版本历史的一部分。随着项目规模扩大，规范的提交信息能够提高版本历史的可读性，也便于协作开发和后续维护。

2. 《语义化版本 2.0.0》

语义化版本（Semantic Versioning, SemVer）使用：MAJOR.MINOR.PATCH三部分共同表示软件的版本状态。

当软件出现不兼容的 API 修改时，增加主版本号；当增加向下兼容的新功能时，增加次版本号；当只进行向下兼容的问题修复时，增加修订号。

语义化版本的价值在于，版本号本身就能够向使用者传递软件变化程度和兼容性信息。在软件依赖较多的项目中，这种规范能够减少升级时的不确定性，并使不同版本之间的关系更加清晰。

3. 为什么要学习 Git

完成本次实验后，我对 Git 的作用有了更加具体的理解。

首先，在个人开发中，Git 可以记录代码的修改历史，使开发者能够查看不同阶段的代码状态，在修改出现问题时也可以回到之前的版本。

其次，在多人协作中，Git 的分支机制允许不同成员在相对独立的开发线上工作。不同修改完成之后，可以通过 merge 将其重新整合到主分支中。当不同开发者修改了同一文件的同一位置时，Git 会检测到冲突，并将最终选择交给开发者处理。

此外，Git 记录的不只是最终代码，也记录了软件开发过程本身。每一次 commit 对应一次明确的修改，Commit Message 用于说明修改目的，branch 表示不同开发任务之间的关系，而软件版本号又进一步描述软件发布阶段和兼容性变化。

本次实验中，从第一次 commit，到创建 feature 分支，再到主动制造并解决 merge conflict，我实际经历了 Git 如何管理不同版本之间的关系。因此，我认为 Git 的价值不仅在于将代码上传到 GitHub，更重要的是帮助开发者有序地记录、管理和协同完成软件开发过程。

四、分支管理、合并与冲突解决

1. 创建 feature 分支

```
yilin@LAPTOP-ET6G27C4:~/lab0-Gitlab$ git switch -c feature
Switched to a new branch 'feature'
yilin@LAPTOP-ET6G27C4:~/lab0-Gitlab$ git branch
* feature
  main
```

2. 在 feature 分支修改并提交

```
yilin@LAPTOP-ET6G27C4:~/lab0-Gitlab$ git status
On branch feature
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working
directory)
        modified:   main.c

no changes added to commit (use "git add" and/or "git commit
-a")
yilin@LAPTOP-ET6G27C4:~/lab0-Gitlab$ git add main.c
yilin@LAPTOP-ET6G27C4:~/lab0-Gitlab$ git commit -m "Modify mi
ain.c on feature"
[feature d7fcddd] Modify miain.c on feature
1 file changed, 1 insertion(+), 1 deletion(-)
```

此时 `feature` 分支已经产生了一个独立于 `main` 分支的新 commit。

3. 在 `main` 分支修改同一位置

```
yilin@LAPTOP-ET6G27C4:~/lab0-Gitlab$ git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
yilin@LAPTOP-ET6G27C4:~/lab0-Gitlab$ git branch
  feature
* main
yilin@LAPTOP-ET6G27C4:~/lab0-Gitlab$ git add main.c
yilin@LAPTOP-ET6G27C4:~/lab0-Gitlab$ git commit -m "Modify ma
in.c on main"
[main f3845d3] Modify main.c on main
1 file changed, 1 insertion(+), 1 deletion(-)
```

为了保证之后执行 merge 时发生冲突，我在 `main` 分支中修改了 `main.c` 中与 `feature` 分支相同的位置，但写入了不同内容。此时 `main` 和 `feature` 从共同的历史节点出发，各自产生了新的 commit，并且两个分支修改了同一文件的同一位置。

4. 合并并产生冲突

```
yilin@LAPTOP-ET6G27C4:~/lab0-Gitlab$ git merge feature
Auto-merging main.c
CONFLICT (content): Merge conflict in main.c
Automatic merge failed; fix conflicts and then commit the res
ult.
```

产生冲突的原因是：`main` 和 `feature` 分支分别修改了 `main.c` 的同一位置，而且修改后的内容不同。Git 无法判断哪一方的代码才是开发者真正希望保留的结果，因此不能自动完成合并。

5. 手动解决冲突

发生冲突后，在 VSCode 中打开 `main.c`，可以看到类似以下结构的冲突标记：

```
<<<<<<< HEAD
main 分支中的内容
=====
feature 分支中的内容
>>>>>>> feature
```

Git 能够检测不同版本之间的冲突，但不能判断哪一段代码在程序语义上更加合理，因此最终版本需要由开发者人工决定。

在本实验中，我手动选择并整理最终代码，使 `main.c` 恢复为正常的 C 源文件。

6. 标记冲突已解决并完成合并提交

在普通开发中，`git add` 表示将修改放入暂存区；而在 merge conflict 场景中，这一步同时表示开发者已经确认当前文件的冲突处理结果。

```
yilin@LAPTOP-ET6G27C4:~/lab0-Gitlab$ git add main.c
yilin@LAPTOP-ET6G27C4:~/lab0-Gitlab$ git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

All conflicts fixed but you are still merging.
  (use "git commit" to conclude merge)

Changes to be committed:
  modified:   main.c

yilin@LAPTOP-ET6G27C4:~/lab0-Gitlab$ git commit -m "Final"
[main 0db35c0] Final
```

至此，`feature` 分支中的开发历史被合并到了 `main` 分支。

7. 查看提交图

```
yilin@LAPTOP-ET6G27C4:~/lab0-Gitlab$ git log --oneline --graph
--all --decorate
*   0db35c0 (HEAD -> main) Final
| \
|  * d7fcddd (feature) Modify miain.c on feature
| * | f3845d3 Modify main.c on main
| /
* 1818a06 (origin/main, origin/HEAD) Complete
* a40c8e9 Initial commit
```

如图可以观察到 `main` 与 `feature` 首先从共同节点分叉，各自产生新的 commit，最后通过 `merge commit` 重新汇合。这说明 Git 在 `merge` 时不会删除某一条分支的开发历史，而是在保留原有 commit 的基础上创建新的合并节点，从而同时保存两个分支的发展过程。

五、实验总结

通过本次实验，我完整实践了 Git 从本地修改、暂存、提交到远程推送的基本流程，并进一步使用分支完成独立开发、合并以及冲突处理。

实验过程中最直观的收获是理解了工作区、暂存区和仓库之间的关系，以及 Git 如何通过 `branch`、`commit` 和 `merge` 管理不同版本。相比单纯记忆 Git 命令，通过实际制造并解决一次 `merge conflict`，我对 Git 的版本管理和多人协作机制有了更加清晰的认识。